

ΕΝΟΤΗΤΑ 7 ΕΠΙΣΤΗΜΟΝΙΚΟΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ

Αλγοριθμική Επίλυση Εξισώσεων ■ Η δύναμη της εκθετικής μετάδοσης

7.1 Εισαγωγή

Στη σημερινή εποχή οι εξισώσεις που σχηματίζονται κατά τη μοντελοποίηση πραγματικών προβλημάτων ή φυσικών φαινομένων έχουν τόσες πολλές μεταβλητές, που η επίλυσή τους με χαρτί και μολύβι είναι πρακτικά αδύνατη. Για παράδειγμα, ο αλγόριθμος που εκτελείται πίσω από το λογισμικό Τεχνητής Νοημοσύνης ChatGPT χρησιμοποιεί δισεκατομμύρια μεταβλητές. Η υπολογιστική επιστήμη (ή επιστήμη του υπολογισμού) ασχολείται με τον σχεδιασμό και την υλοποίηση υπολογιστικών μοντέλων για την επίλυση προβλημάτων μέσω υπολογιστή. Με την αλματώδη εξέλιξη της Πληροφορικής υπάρχουν σήμερα υπολογιστικά συστήματα, με τα οποία μπορούμε να υλοποιήσουμε εξελιγμένα υπολογιστικά για την πρόβλεψη του καιρού και την αποκωδικοποίηση του ανθρώπινου γονιδιώματος. Πλέον, η υπολογιστική επιστήμη χρησιμοποιείται στην Ιατρική, τη Βιολογία, τη Χημεία και τη Φυσική. Ένα πρόσφατο παράδειγμα είναι η ανάπτυξη ενός αλγορίθμου βασισμένου σε νευρωνικά δίκτυα και υλοποιημένου στη γλώσσα Python, ο οποίος δημιούργησε την πρώτη εικόνα μιας μαύρης τρύπας. Ο αλγόριθμος τροφοδοτήθηκε από 5 PetaBytes δεδομένων που κατέγραψαν επτά μεγάλα τηλεσκόπια.

Στην ενότητα αυτή θα παρουσιάσουμε δυο παραδείγματα απλών υπολογιστικών μοντέλων για την επίλυση προβλημάτων στα Μαθηματικά και τη Βιολογία.

7.2 Αλγοριθμική επίλυση εξισώσεων

Υπάρχουν προβλήματα σε όλες τις επιστήμες στα οποία εμπλέκονται εκατομμύρια ή και δισεκατομμύρια μεταβλητές. Αυτό καθιστά πρακτικά αδύνατη την επίλυσή τους με συμβατικές μεθόδους. Γι' αυτό αναπτύσσουμε υπολογιστικά μοντέλα για την επίλυση αυτών των προβλημάτων, αξιοποιώντας την τεράστια ισχύ των σημερινών υπολογιστικών συστημάτων.

Ως εκ τούτου έχουν αναπτυχθεί αλγόριθμοι υπολογισμού των λύσεων όλων των τύπων των εξισώσεων. Ποιος αλγόριθμος νομίζετε ότι είναι ο πιο απλός για την επίλυση μιας εξίσωσης; Μα, φυσικά, ο διεξοδικός έλεγχος όλων των πιθανών λύσεων μέχρι να βρούμε τη σωστή. Δοκιμάζουμε δηλαδή έναν-έναν τους αριθμούς για να δούμε ποιος ικανοποιεί την εξίσωση, μέχρι να βρούμε τη λύση.

Ας υποθέσουμε, για παράδειγμα, ότι θέλουμε να λύσουμε την εξίσωση: $10x + 5 = 105$. Εργαζόμαστε ως εξής:

$$\text{Για } x = 1 \text{ έχουμε } 10 \cdot 1 + 5 = 15 \neq 105$$

$$\text{Για } x = 2 \text{ έχουμε } 10 \cdot 2 + 5 = 25 \neq 105$$

$$\text{Για } x = 3 \text{ έχουμε } 10 \cdot 3 + 5 = 35 \neq 105$$

.....

$$\text{Για } x = 10 \text{ έχουμε } 10 \cdot 10 + 5 = 105$$

Θα ήταν εξαιρετικά χρονοβόρο και αντιπαραγωγικό για μας να ελέγχουμε μια-μια όλες τις πιθανές λύσεις μιας εξίσωσης μέχρι να μας χαμογελάσει η τύχη και να βρούμε τη σωστή, ειδικά για πολύ μεγάλους αριθμούς. Για παράδειγμα, αν η λύση της εξίσωσης είναι ο αριθμός $360 \cdot 10^6$ και υποθέσουμε ότι μπορούμε να εκτελούμε έναν έλεγχο ανά 10 δευτερόλεπτα, θα χρειαστούμε $3600 \cdot 10^6$ δευτερόλεπτα, δηλαδή 10^6 ώρες, ή αλλιώς 114 χρόνια!

Αν, όμως, κωδικοποιήσουμε τον παραπάνω απλό αλγόριθμο σε μια γλώσσα προγραμματισμού, τότε μπορεί να εκτελεστεί ακόμα και στο κινητό μας τηλέφωνο. Σε αυτή την περίπτωση, θα χρειαστεί λιγότερο από ένα δευτερόλεπτο για να μας δώσει την απάντηση, αφού ένα μέσο κινητό τηλέφωνο μπορεί να εκτελέσει πάνω από 1 δισεκατομμύριο πράξεις το δευτερόλεπτο.

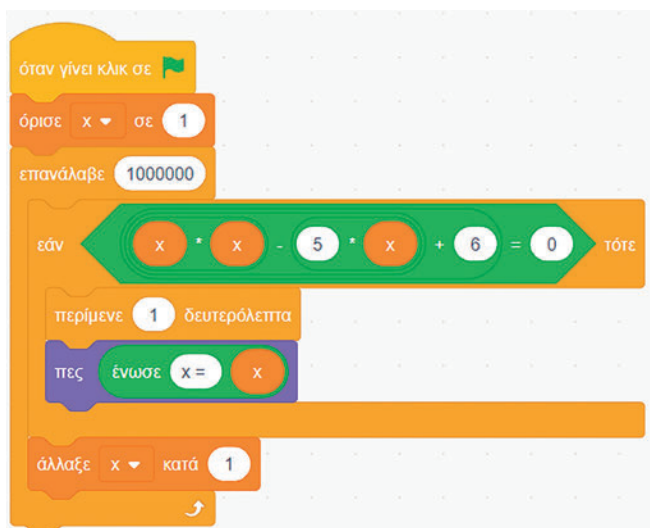
Για κάθε αριθμό από 1 μέχρι 1000000
 Αν ο αριθμός είναι λύση της εξίσωσης $10x+5=100$ τότε
 Γράψε τον αριθμό
 Τέλος_αν
 Τέλος_επανάληψης

Εικόνα 7.3 Αλγόριθμος επίλυσης της εξίσωσης $10x + 5 = 105$ με ωμή βία (brute force).

Η παραπάνω αναπαράσταση αλγορίθμου είναι σε μια ψευδο-γλώσσα την οποία χρησιμοποιούμε για ένα πρώτο σκαρίφημα του αλγορίθμου που δεν είναι αυστηρά διατυπωμένο και, άρα, μη εκτελέσιμο. Για να εκτελέσουμε τον παραπάνω αλγόριθμο σε υπολογιστή, θα πρέπει να τον διατυπώσουμε σε μια γλώσσα προγραμματισμού όπως η Python.

Με αυτή τη στρατηγική της εξαντλητικής αναζήτησης, που είναι γνωστή και ως ωμή βία (brute force), μπορούμε να λύσουμε οποιαδήποτε εξίσωση με ακέραιες λύσεις, για παράδειγμα την $x^2 - 5x + 6 = 0$.

```
for x in range(1, 1000001):
    if 10*x+5==100:
        print( "x = ", x )
```



```
for x in range(1, 1000001):
    if x*x-5*x+6 == 0:
        print( "x = ", x )
```



Δραστηριότητα 1

Τροποποιήστε το παραπάνω πρόγραμμα ώστε να λύνει τις εξισώσεις:

α) $26x - 260 = 0$ β) $10x^2 - 50x + 60 = 0$ γ) $x^3 - 9x^2 + 26x - 24 = 0$



Δραστηριότητα 2

Τροποποιήστε το παραπάνω πρόγραμμα ώστε να λύνει τις εξισώσεις:

α) $4x + 20 = 0$ β) $2x - 1 = 0$ γ) $4x - 1 = 0$ δ) $x^2 - 2 = 0$

Τι πρόβλημα παρατηρείτε κατά την εκτέλεσή τους;
 Γιατί συμβαίνει αυτό;
 Ποια λύση προτείνετε σε κάθε περίπτωση;



Δραστηριότητα 3

Να διερευνήσετε γιατί στα παραπάνω προγράμματα στη γλώσσα Python η εντολή επανάληψης έχει άνω όριο 1.000.001 και όχι 1.000.000.

Στο περιβάλλον προγραμματισμού Thonny ή σε κάποιον online διερμηνευτή της Python δοκιμάστε να εκτελέσετε τα παρακάτω προγράμματα και σχολιάστε τα αποτελέσματά τους.

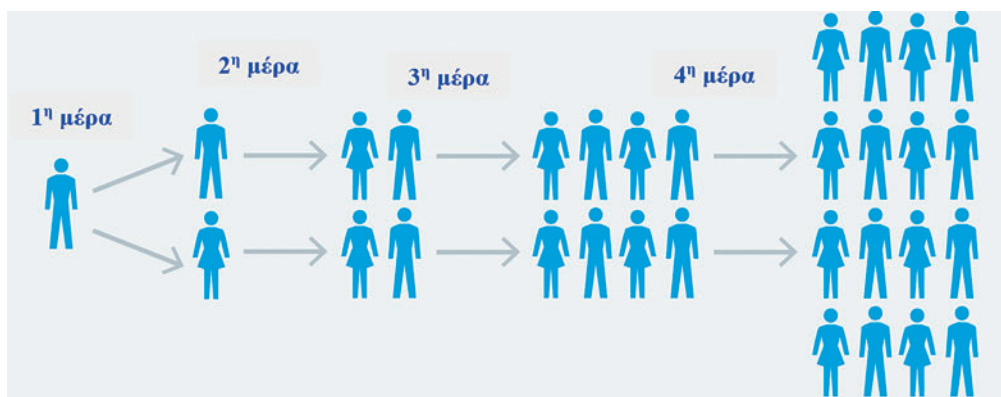
```
for x in range(1, 1000001):
    if x*x-5*x+6 == 0:
        print( "x = ", x )
```

```
for x in range(5, 12):
    print( "x = ", x )
```

```
for x in range(12):
    print( "x = ", x )
```

7.3 Η δύναμη της εκθετικής μετάδοσης

Μια σημαντική σύγχρονη πρακτική εφαρμογή των αλγορίθμων είναι η μελέτη φαινομένων μέσω υπολογιστικής μοντελοποίησης. Η μαθηματική μοντελοποίηση που χρησιμοποιείται για μια ασθένεια μπορεί να συμβάλει στην πρόβλεψη της προόδου μιας νόσου και να δείξει την πιθανή έκβαση της εξάπλωσής της. Οι επιστήμονες μελετούν μαθηματικά μοντέλα μετάδοσης ιών, ώστε να πάρουν τις σωστές αποφάσεις για τις παρεμβάσεις που θα γίνουν. Η πολυπλοκότητα των μοντέλων απαιτεί την χρήση υπολογιστή για την αποτελεσματική διερεύνησή τους, μέσω της υλοποίησής τους σε κάποια γλώσσα προγραμματισμού. Ο τομέας της Πληροφορικής που ασχολείται με την υλοποίηση αυτών των μοντέλων είναι γνωστός στην βιβλιογραφία ως *επιστημονικός προγραμματισμός* (scientific programming). Αρκετοί ερευνητές χρησιμοποιούν, επίσης, τον όρο *υπολογιστική επιστήμη* (computational science).



Ένας μεταδοτικός ιός έχει τα εξής χαρακτηριστικά:

- ▶ Κάθε άνθρωπος που έχει μολυνθεί από τον ιό μπορεί να τον μεταδώσει την επόμενη μέρα σε 2 ακόμα ανθρώπους με τους οποίους έχει έρθει σε επαφή.
- ▶ Κάθε άνθρωπος είναι μεταδοτικός μόνο για μια ημέρα, για την επόμενη της μόλυνσης.

Θέλουμε να σχεδιάσουμε ένα υπολογιστικό μοντέλο με βάση το οποίο θα μπορούμε να υπολογίζουμε εύκολα πόσα κρούσματα έχουμε κάθε μέρα και πόσα συνολικά.

Αρχικά, θα χρειαστεί να διερευνήσουμε το φαινόμενο για μικρούς αριθμούς, με σκοπό να γενικεύσουμε με τη χρήση κάποιας μεταβλητής. Θα χρειαστεί να συμπληρώσουμε τον παρακάτω πίνακα:

Αριθμός ημέρας	Ημερήσια Κρούσματα	Συνολικά κρούσματα
1	1	1
2	2	$1 + 2 = 3$
3	4	$1 + 2 + 4 = 7$
4		
5		
6		
8		
10		
.....		
20		
N		

1. Σε πόσες μέρες θα έχουν μολυνθεί όλοι οι κάτοικοι της χώρας μας, που είναι περίπου 10^7 ;
2. Να γράψετε ένα πρόγραμμα το οποίο θα δέχεται το πλήθος των ημερών, και θα εμφανίζει πόσοι θα έχουν μολυνθεί μετά από ένα μήνα.
3. Καθώς υπολογίζετε το πλήθος των μολυσμένων ανθρώπων σε κάθε βήμα, σκεφτείτε ποιες πράξεις επαναλαμβάνονται κάθε φορά και τι μεταβλητές χρειάζεστε.
4. Τι αλλαγές πρέπει να κάνετε στο πρόγραμμα, αν εμφανιστεί μια νέα μετάλλαξη του ιού η οποία είναι δύο φορές πιο μεταδοτική; Δηλαδή κάθε άνθρωπος μολύνει σε μια μέρα τέσσερις συνανθρώπους του;
5. Πόσες μέρες χρειάζονται τώρα για να μολυνθεί όλη η χώρα με τη νέα μετάλλαξη;
6. Πόσες μέρες/μήνες χρειάζονται για να μολυνθεί ο πληθυσμός ολόκληρου του πλανήτη με αυτή τη νέα μετάλλαξη; Συμφωνεί αυτή η πρόβλεψη με την πραγματικότητα; Γιατί; Τι δεν έχει προβλεφθεί στο μοντέλο;

Παρακάτω δίνονται δυο ενδεικτικά προγράμματα, σε Scratch και Python, τα οποία υλοποιούν το υπολογιστικό μοντέλο που περιγράφεται παραπάνω:



```

μέρες = int(input("Δώσε τον αριθμό των ημερών: "))
Ημερήσια_κρούσματα = 1
Συνολικά_κρούσματα = 0
for i in range(μέρες):
    Συνολικά_κρούσματα += Ημερήσια_κρούσματα
    Ημερήσια_κρούσματα = 2*Ημερήσια_κρούσματα

print("Συνολικά νόσησαν = ", Συνολικά_κρούσματα)

```

7.4 Άσκηση

1

Να δημιουργήσετε τα αντίστοιχα υποπρογράμματα για τα υπολογιστικά μοντέλα της επίλυσης εξισώσεων και της εκθετικής μετάδοσης του ιού. Ποιές παραμέτρους χρειάζεται κάθε υποπρόγραμμα;